

Mohammed Zakaria Mutlaq

Data Engineer

mohamedzakaria.cs@gmail.com | +20 150 101 5701 | Giza, Egypt | Military: Exempted
[linkedin.com/in/eng-zakaria](https://www.linkedin.com/in/eng-zakaria) | github.com/Eng-Zakaria | eng-zakaria.medium.com | eng-zakaria.github.io/Portfolio

SUMMARY

Data Engineer with 1.5+ years of hands-on experience building and automating end-to-end data pipelines across batch and real-time systems. Specializes in dimensional modeling, ETL/ELT pipeline development, and scalable analytics platforms including data warehouses, star schemas, and Spark/Airflow orchestration. Production experience on AWS (EC2, S3, IAM, VPC) with exposure to Azure, plus a track record of implementing data quality checks, monitoring, and validation frameworks that improved reporting accuracy and eliminated pipeline failures.

PROFESSIONAL EXPERIENCE

Data Engineer Intern, Waffarha

07/2026 to Present
Cairo

- Built a bilingual (Arabic/English) RAG chatbot pipeline for Waffarha's offers/FAQ data, serving through a FastAPI backend and Streamlit UI, with Redis-backed session handling to support concurrent multi-user querying.
- Evaluated 4 multilingual embedding models (multilingual-E5-large, multilingual-E5-base, BGE-M3, mpnet) for retrieval quality, selecting multilingual-E5-large for production based on 94.9% recall and 0.836 MRR across Arabic, English, and Franco-Arabic queries.
- Benchmarked 5 vector store backends (FAISS, Chroma, Qdrant, LanceDB, pgvector) on 18k+ indexed documents; FAISS delivered the fastest retrieval (0.62ms avg. query latency vs. 7.6 to 65ms for the alternatives) while all backends held 85%+ recall, informing the production backend choice.
- Designed a hybrid intent-routing engine that resolves ~75% of queries through deterministic, sub-100ms direct-answer shortcuts (FAQ/offer template matching) ahead of LLM generation, combining dense embeddings with a lexical token-overlap bonus and cosine-similarity score normalization across all 5 vector backends; also evaluated cross-encoder reranking against this approach and kept the simpler hybrid in production based on the comparison. Personal (ClickHouse) and catalog queries are routed away from the vector index entirely to keep results fresh and PII out of the embedding store.
- Built an automated fact-checking layer that extracts verifiable claims (prices, discounts, dates) from LLM-generated answers and cross-checks them against retrieved sources, stripping unsupported claims to curb hallucination in the 1.5B to 3B parameter fallback models.
- Automated Airflow DAGs to schedule ingestion from MySQL and ClickHouse, deduplicate by content hash, version structured records (SCD2), and trigger re-embedding and vector index updates on new or changed data.
- Ran a systematic LLM evaluation across 8 dense and Mixture-of-Experts models (Qwen2.5, Granite3.1-MoE, OLMoE, Aya-Expanse, Command-R7B-Arabic, DeepSeek-V2, Llama3.2), tuning inference configs (temperature, top-p, context window) per model; tuned a compact ~2GB MoE model to run reliably in production at 88+ tokens/sec without sacrificing answer quality.
- Built an internal Python SDK exposing the RAG pipeline as both a client library and an API layer, standardizing access to the vector store and embedding logic for other services, cutting integration bugs and duplicated retrieval code, and letting other engineers onboard against a single stable interface instead of the retrieval internals. Containerized the full stack (FastAPI, Streamlit, Airflow, vector DB, Redis) with Docker Compose, baking the Ollama and embedding models into the images at build time for zero-runtime-network production deployment, and enabling one-command local deployment with seamless vector-index sharing between the DAG and the chat app.
- Built a CLI tool to bootstrap the project environment, validating system requirements, installing models and dependencies, and querying the Hugging Face registry plus a custom model-JSON catalog to recommend suitable embedding/LLM candidates.
- Built a CI-integrated evaluation harness (110+ test cases across 7 categories: offers, FAQs, personal queries, follow-ups, catalog, negative/hallucination, and Arabic) with reusable, parameterized scripts (CLI args for LLM, embedding model, vector store, and config selection) for automated cross-matrix benchmarking; current baseline holds a 96.4% pass rate (99% direct-answer precision) within defined per-category latency budgets, with exit-code gating to block regressions in CI.
- Designed and built a modular market-intelligence pipeline monitoring public mentions of Waffarha across App Store, Google Play, Reddit, Google News, and web search, using a connector-registry architecture where adding a new source requires only one new connector file, with no changes to ingestion, analysis, storage, or the dashboard.
- Built a multilingual sentiment and topic-tagging layer for the market-intelligence pipeline, routing Arabic, English, and Arabizi (Arabic in Latin script) mentions to language-specific models (AraBERT for Arabic, RoBERTa for English) and a rule-based tagger across 6 business categories (app bugs, customer service, pricing, wallet security, offer accuracy, delivery), surfaced through a Streamlit dashboard with sentiment trends, topic breakdowns, and a negative-mention triage feed.
- Designed and built an automated partner price-integrity pipeline detecting cases where commercial partners undercut their own Waffarha voucher prices on their websites, orchestrated as a weekly partner-reachability audit plus a daily 6-stage ingest -> scrape -> match -> compare -> report cycle deployed via two Airflow DAGs.
- Built a tiered product-matching system (exact product code -> manual override -> fuzzy text match via rapidfuzz) feeding a dual scraping engine (BeautifulSoup for static pages, Playwright for JS-rendered sites) with resumable per-partner JSON checkpoints, plus a canonical reporting layer generating HTML, Excel, and Markdown violation reports with configurable tolerance and watch-list bands (default 2%).
- Built a local ClickHouse sync tool mirroring production tables for safe dev/test iteration without touching production data, plus a

comparison harness benchmarking RAG-retrieval vs. direct ClickHouse catalog-query latency and answer quality to inform query-routing decisions.

Big Data Engineering Intern, National Telecommunication Institute (NTI)

04/2026 to 07/2026
Smart Village

- Built and automated ETL data processing workflows using Spark, Python, and SQL, orchestrating multi-step pipelines end-to-end with Airflow.
- Processed large-scale datasets on Hadoop distributed systems (HDFS, MapReduce, YARN); worked with HBase (NoSQL), Hive (data warehousing), and InfluxDB (time-series).
- Built data ingestion and streaming workflows using Kafka, Flume, and Sqoop, moving data between relational sources and the Hadoop ecosystem.
- Gained hands-on experience with AWS (EC2, S3, IAM, VPC) for cloud-based data solutions; built monitoring dashboards and reports in Power BI, Tableau, and Grafana.

Data Engineer, Nexlify Startup

01/2025 to 01/2026
Cairo

- Built an ETL pipeline with Airflow to ingest data from heterogeneous sources like Excel sheets and clinic system APIs across 10+ clinics, covering customers, staff, revenue, orders, schedules, appointments, and sales at both clinic-level and company-wide levels.
- Designed and built a centralized data warehouse from scratch, evaluating MySQL vs. PostgreSQL for the storage engine, using dimensional modeling with star schemas, fact/dimension tables, and junction (bridge) tables to resolve many-to-many relationships; adapted the star/snowflake schema design as requirements evolved across Medical, IoT, and Fintech domains, integrating 10+ disparate sources (API logs, CSV files, Excel) and accelerating reporting cycles by 40%.
- Engineered the pipeline to reliably process daily transactional volumes at scale (50+ new customer records and 200 appointments per day, similar order volumes, and 1,000 revenue/ledger records daily), consolidating data across 10+ clinics, including logic to unify even unrelated clinics' data without conflicts.
- Modeled multi-role staff assignments, where a single staff member could hold more than one role, ensuring accurate attribution and consistency in downstream reporting.
- Performed data mapping between source systems and the data warehouse, defining fact and dimension structures, keys, and relationships used downstream for reporting and application consumption.
- Refactored legacy SQL pipelines using CTEs and optimized JOIN logic across PostgreSQL, SQL Server, MySQL, and SQLite; achieved a 30% reduction in query latency and eliminated 100% of dashboard timeouts for the BI team.
- Cleaned and standardized millions of historical records using custom Python automation scripts, handling mixed Arabic/English text and numeric data formats, and enforcing data quality and integrity before loading into the warehouse.
- Built the analytics layer and authored Power BI reports/dashboards designed around business KPIs on top of the unified warehouse, using a clinic-wise switcher/filter so a single report model served every clinic and company-wide views, instead of maintaining a separate report per clinic, giving stakeholders the visibility to support data-driven decisions.

AI Instructor, ISchool

06/2024 to 11/2024
Cairo

- Simplified complex data and ML concepts for 100+ students, leading hands-on sessions on model evaluation with a 90% project completion rate.
- Facilitated technical labs and sessions, ensuring 100% of curriculum modules were completed and documented on schedule.

Blockchain Analysis Intern, UZH (University of Zurich)

06/2023 to 08/2023
Zurich, Switzerland

- Extracted and processed large-scale Web3/Fintech datasets from the Ethereum blockchain using Python for research.
- Developed automated validation scripts to sanitize raw node data (blocks/transactions), improving downstream data integrity.
- Generated on-chain data summaries and quality reports for stakeholder visibility into data health and trends.

EDUCATION

Bachelor's in Computers and Artificial Intelligence, Helwan University

09/2020 to 05/2024
Cairo

GPA: 3.3 | Graduation Project Grade: A+

TECHNICAL SKILLS

Programming & Scripting: Python, NumPy, Pandas, PySpark, Scala, SQL, Bash

Big Data Distributed Computing & Processing: Apache Spark, Flink, Hadoop (HDFS / YARN / MapReduce)

Data Ingestion & Streaming: Apache Kafka, NiFi, Sqoop, Flume

ETL / Pipeline Orchestration: Apache Airflow, AWS Glue (exposure)

Cloud & DevOps: AWS (S3 / IAM / EC2 / VPC), Azure (Blob Storage), Docker, Git/GitHub, Linux

Data Warehousing & OLAP: Apache Hive, HBase, Dimensional Modeling (Star / Snowflake), ClickHouse

Data Quality & Lifecycle: Data Validation & Cleansing, SCD2 Versioning, Data Lifecycle Management

Databases: PostgreSQL, SQL Server, MySQL, InfluxDB, Neo4j, Elasticsearch

Data Visualization & Analysis: Power BI, Tableau, Grafana, Matplotlib, Seaborn
Soft Skills: Time Management, Communication, Problem Solving, Teamwork, Quick Learner

TRAINING & CERTIFICATES

- Huawei HCIP | Big Data (H13-723), National Telecommunication Institute (NTI), 2026
- AWS Certified Cloud Practitioner | 2026, AWS
- ITI Full Stack Development Trainee | 2021, ITI: Architected and deployed end-to-end C# and .NET web applications, from database schema design to front-end UI; delivered 3 production-ready demos ahead of schedule.

PROJECTS

Graduation Project: Hadith Chat App

Django, Qdrant, NLP/RAG

- Built an Arabic NLP, RAG-based chat app using Django and Qdrant.
- Reduced AI hallucination by improving data quality and fine-tuning retrieval, achieving chatbot accuracy of 70%.

Aircraft Tracking Pipeline

Spark, Kafka, Docker, Flume, InfluxDB, Grafana, HDFS

- Built a scalable, containerized real-time streaming pipeline ingesting live aircraft state vectors from the adsb.lol ADS-B API, using dual Flume agents (HTTP -> Kafka, Kafka -> HDFS) and Spark Structured Streaming (Kafka -> InfluxDB).
- Modeled two InfluxDB measurements (raw per-aircraft state + 1-minute windowed aggregates) and built live Grafana dashboards (geomap, count/speed/altitude by aircraft type) while archiving raw streams to HDFS.

Reddit Spark Pipeline

PySpark, Reddit API, Parquet

- Built a 7-stage PySpark batch ETL pipeline (Bronze/Silver/Gold medallion architecture) ingesting posts from 10 subreddits via the Reddit public API, with data quality filtering, engagement scoring, and topic-tagging UDFs.
- Implemented window functions for engagement ranking and score velocity, broadcast joins for category enrichment, and partitioned Parquet output optimized for downstream analytics.

HDFS Manager GUI

Python, Tkinter

- Built a dual-pane desktop GUI (dark-themed, Tkinter) for managing HDFS alongside the local filesystem, including service start/stop for HDFS/YARN with live daemon monitoring, multi-select PUT/GET transfers, and an in-app file viewer (cat/tail -f/hexdump/search).
- Implemented a threaded architecture so all HDFS/shell operations run off the UI thread, keeping the app responsive during long-running transfers and live log tailing.

Fraud Detection

Python, Kafka, Docker, Airflow, FastAPI, MLflow

- Developed a real-time streaming architecture using Kafka and Airflow to ingest, clean, and extract features from high-volume transaction data with sub-second latency.
- Containerized a full-stack monitoring ecosystem with Docker, integrating a Plotly/Dash dashboard and multi-channel alerting (Slack/Email) to track model drift and system health.

Credit Risk Analysis

Streamlit, Plotly, Docker, Airflow, FastAPI, MLflow

- Automated end-to-end ML lifecycle orchestration using Apache Airflow DAGs, integrating BigQuery-compatible SQL data marts and feature stores.
- Deployed a scalable fraud-scoring microservice via FastAPI and Scikit-learn, leveraging MLflow for automated model tracking and Redis for high-performance data lookups.

Blockchain Analyzer

Python, Web3.py, Spark, Neo4j, Docker

- Built a blockchain forensics platform comparing Neo4j (graph) vs. PySpark (batch) approaches for Ethereum transaction analysis, with a Bronze -> Silver -> Gold ETL pipeline and Elasticsearch/Kibana dashboards.
- Benchmarked both engines across wallet-degree, multi-hop traversal, and suspicious-pattern-detection operations to produce a data-backed recommendation on when to use each.